KlikEngine – визуальный редактор кода

Оглавление

Глава 1 О программе 2
Глава 2 Процесс установки 4
Установка на Windows 4
Установка на Linux 5
Глава 3 Работа с BlocklyKlik 5
Структура редактора 5
Горячие клавиши 6
Программирование 6
Переменные и типы данных 6
Работа с пинами контроллера 7
Блоки математики 8
Ветвление 8
Циклы 10
Временные интервалы 13
Комментарии 13
Кодировка 14
Поиск устройства по i2C 16
Ультразвуковой датчик расстояния 17
Сервомотор 18
Моторы с энкодерами 19
IR-модуль 25
Bluetooth 28
Датчик цвета 31
Дисплей 16x8 33
Двойной датчик линии, кнопка 36
Функции 36
Глава 4 Работа с KLIK EDITOR MICROPYTHON 38
Структура интерфейса 38
Программирование 38

Глава 1 О программе

KlikEngine – это менеджер по управлению двумя редакторами для контроллера **К1** и **К2** наборов КЛИК и НикиРобот.

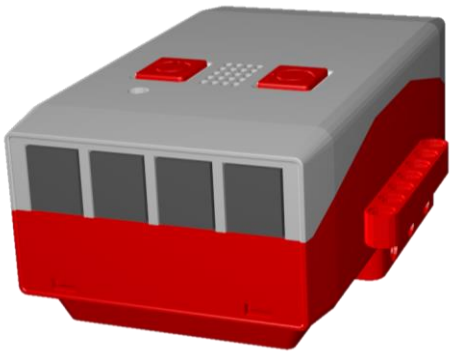


Рисунок 1.1 Контроллер К1



Рисунок 1.2 Контроллер К2

Для программирования К1 используется программа **BlocklyKlik** (на данный момент версии 2.1)



Рисунок 1.3 BlocklyKlik 2.1

Для управления файлами внутри контроллера К2 и программирования используется программа KLIK EDITOR MICROPYTHON

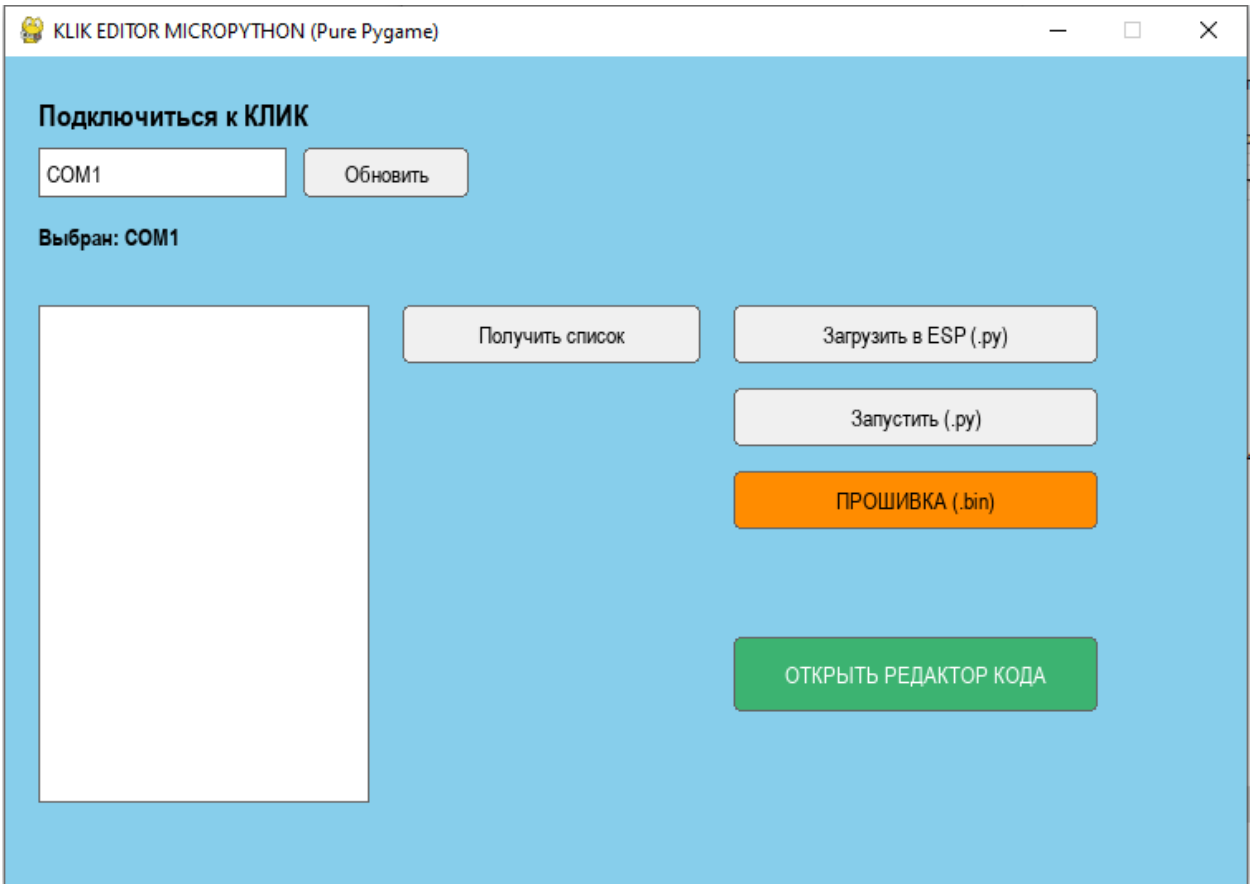


Рисунок 1.4 Klik Editor MicroPython

Оба этих редактора расположены в папке KlikEngine и могут вызываться независимо друг от друга.

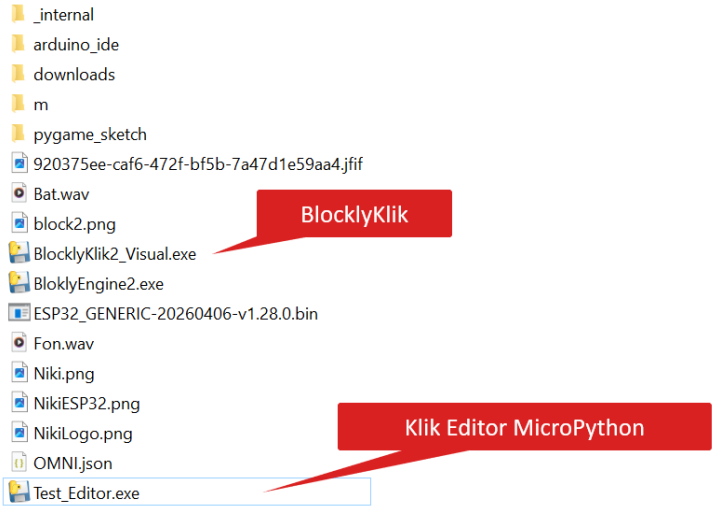


Рисунок 1.5 Содержание папки KlikEngine

Альтернативный способ их вызова – это использовать оконную программу BlocklyEngine2, которая также расположена в папке KlikEngine. Она имеет доступ сразу к двум этим редакторам.

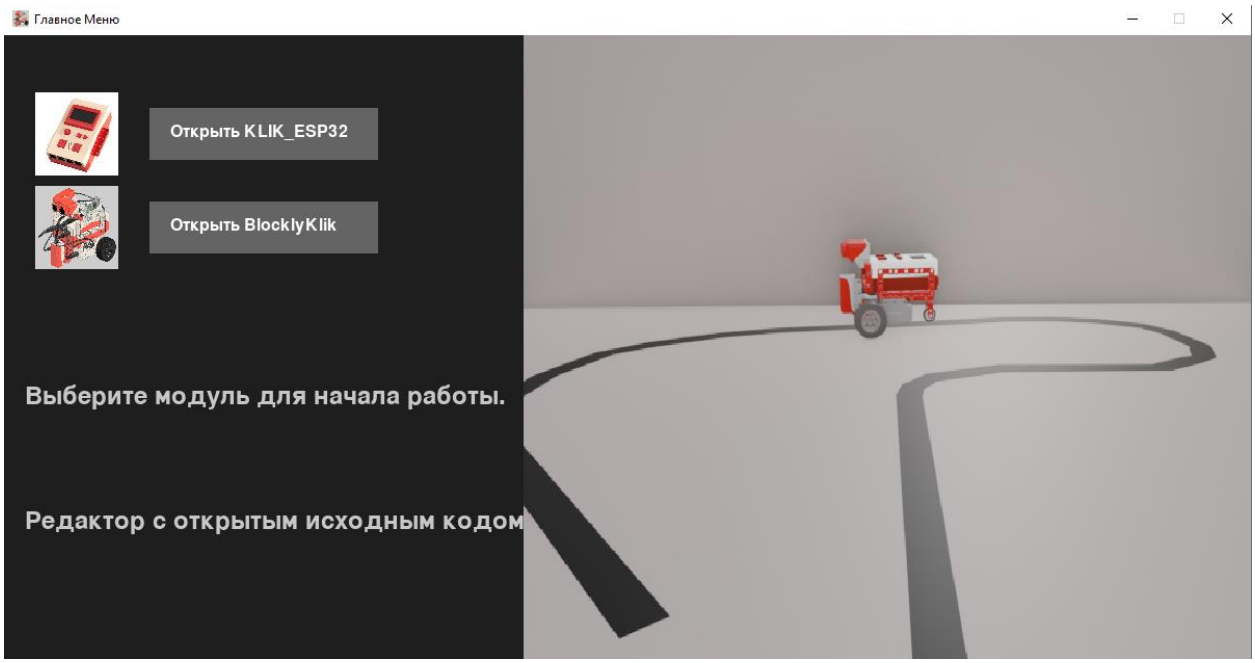


Рисунок 1.6 KlikEngine

Глава 2 Процесс установки

Установка на Windows

Разработанный софт совместим с Windows 8-11. Для его использования достаточно перейти в репозиторий BlocklyKlik на GitHub: https://github.com/Antipat/BlocklyKlik и Выбрать ссылку Windows в файле Readme.

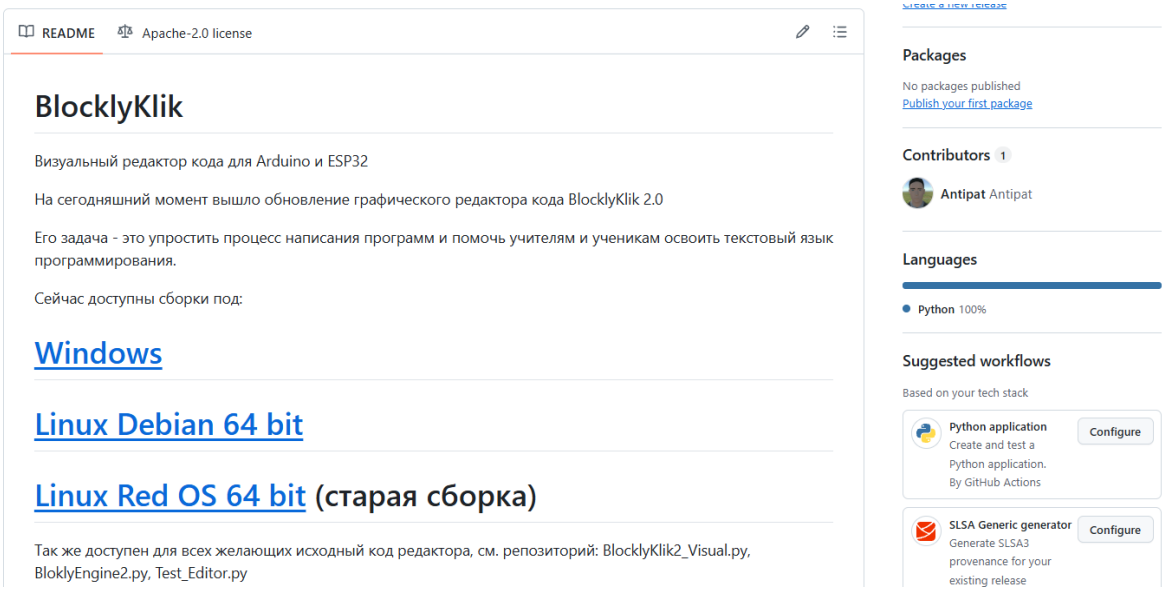


Рисунок 2.1 Ресурсы редактора на GitHub

Скаченный архив необходимо распаковать. В папке KlikEngine будут доступны три исполняемых файла. В зависимости от работы с тем или иным контроллером используйте нужный софт (см. Глава 1).

Установка на Linux

Разработанный софт собран в пакет **.deb**. Тем самым он автоматически совместим с Linux системами семейства Debian. Обязательным параметром является 64 битная разрядность, см. рисунок 2.1.

Если ваша ОС Linux принадлежит другому семейству, то скорее всего в ней отсутствует менеджер пакетов и он может быть не совместим. Тогда вам нужно использовать старую версию сборки редактора или применить утилиту по конвертации **deb** в ваш формат, например **rpm**.

Процедура установки такая же как для Windows:

- Скачать архив с редактором
- Распаковать его
- Установить, используя команду в терминале: **sudo dpkg -i** имя_пакета.deb

Глава 3 Работа с BlocklyKlik

Структура редактора

Редактор разделён на четыре основных окна: программные блоки, рабочее пространство, интерпретация кода на C++ и окно администрирования (рис. 3.1).

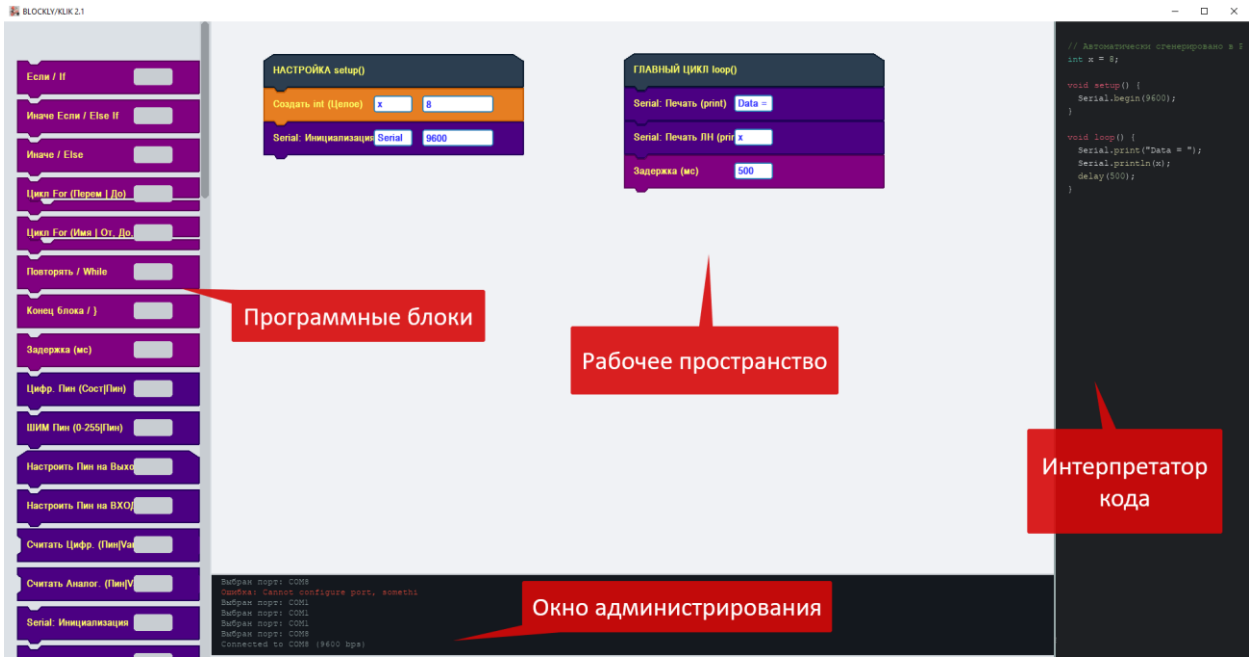


Рисунок 3.1 Интерфейс BlocklyKlik

С окна «**Программные блоки**» можно перетаскивать командные блоки ЛКМ (левой кнопкой мыши) на рабочее пространство. Из них формируется программа для контроллера К1.

В окне «**Рабочее пространство**» создаётся программа из командных блоков. Выполняется только та программа, которая привязана к главным блокам «**setup**» и «**loop**». Программа в разделе **setup** выполняется один раз во время запуска контроллера К1. Здесь, обычно, инициализируются переменные и целые классы из библиотек, а также пины, которые будут задействованы в программе. В разделе loop код будет выполняться циклично всё время пока подаётся питание в контроллере К1.

Окно «**Интерпретатор кода**» конвертирует программу из блоков в набор команд на языке C++, что удобно для тестирования непосредственно в программе Arduino ide. Конвертация происходит путём нажатия клавиши **Space** (пробел), в результате чего отображается код в окне и формируется файл формата *.ino* в папке *pygame_sketch*, где располагаются сами редакторы.

Окно «Администрирования» имеет консоль для возможности отображения получаемых значений с датчиков или программных ошибок. Здесь же содержится панель управления: выбор COM-порта подключаемого контроллера, настройка скорости вывода информации, возможность сохранения и открытие сохранённого файла проекта. А также можно очистить переполненную консоль.

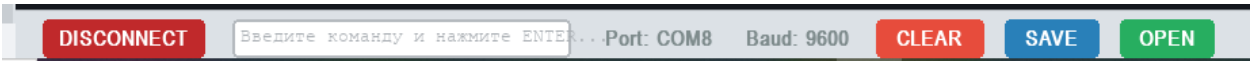


Рисунок 3.2 Панель управления

Горячие клавиши

Редактор поддерживает команды с клавиатуры:

- **Space** – конвертация кода из блоков в C++;
- **Ctrl+C** и **Ctrl+V** – функции копирования и вставки. Работают в рабочем пространстве редактора;
- **L** – компиляция полученного кода в контроллер К1.

Замечание: компилировать код можно только после конвертации его в C++.

Программирование

Программа пишется с помощью графических блоков.

Переменные и типы данных

Для хранения данных в памяти контроллера и дальнейшем их использовании в программе применяют переменные. Они могут хранить данные разного типа: целые числа, дробные числа, строки и массивы.



Рисунок 3.3 Переменные

Работа с пинами контроллера

От контроллера выведены шесть универсальных портов и два под управления DC-моторами. К каждому порту подведены два пина (смотри Приложение).

Для работы с пинами используются такие блоки как на рис. 3.4. С помощью них мы назначаем характер работы пина на приём команд или передачу ими данных на контроллер (OUTPUT и INPUT). Пины могут принимать и выдавать цифровые сигналы (HIGH (1) или LOW (0)), либо аналоговые данные ((0, 255) или (0, 1024)). Для вывод данных с пинов в консоль (терминал) применяют блоки Serial.

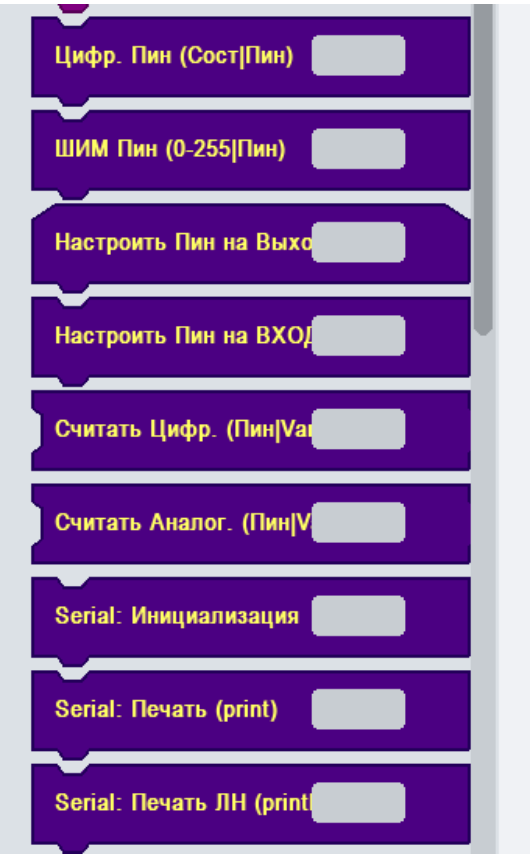


Рисунок 3.4 Блоки управления пинами



Рисунок 3.5 Пример применения блоков работы с пинами.

Блоки математики

Если в программе требуются математические вычисления, то редактор предоставляет набор блоков с распространёнными математическими операторами: сложение, вычитание, деление, возведение в квадрат, назначения констант **π** и **e**, а также нахождения десятичного и натурального логарифма. Дополнительным оператором здесь является конкатенация строк. Примеры реализации представлены на рис. 3.6.

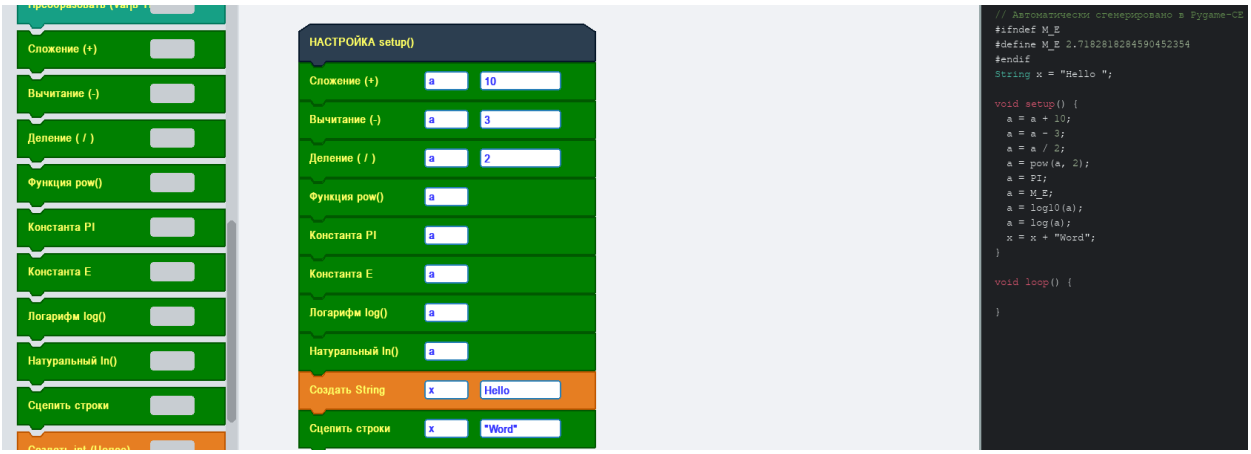


Рисунок 3.6 Математические блоки

Ветвление

В большинстве случаях, программа будет содержать алгоритмы ветвления. В редакторе они представлены тремя основными блоками + блок закрытия скобок.



Рисунок 3.7 Три блока ветвления

Для Arduino ide этот код выглядел бы так:

```
int x = 100;

void setup() {
  if(x<100) {
  } else if (x>100) {
  } else {
  }
}
```

Обратите внимание, на последний блок закрытия алгоритма ветвления. В данном случае он необходим. Проверяйте количество скобок после создания кода и нажатия клавиши Space. В текстовой интерпретации должно быть открытых скобок столько же сколько и закрытых.

Для блока «**Иначе/Else**» ячейку оставляют пустой, она никак не влияет на ход алгоритма.

То, что указано выше – это конструкция условия с полным набором операторов, но мы можем всегда убрать последующих два либо сразу оба, либо любой из них. Так же учтите, что в реальном коде необходимо прописать действие для выполнения каждого условия, иначе в таком виде программа вам выдаст ошибку.

Ниже представлен пример корректной формы записи программы полного ветвления. Вместо вывода в терминал, можно назначать действие на пины.



Рисунок 3.8 Корректная форма записи полного ветвления

Если сгенерировать код в C++, то получим такую запись:

```
int x = 100;

void setup() {
  Serial.begin(9600);
  if(x<100) {
  Serial.println("Малое число");
  } else if (x>100) {
  Serial.println("Большое число");
  } else {
  Serial.println("То что надо");
  }
}
```

Циклы

В редакторе представлены блоки для создания трёх типов циклов.

Первый тип цикла

Конечный стандартный цикл от нуля к указанному положительному значению с шагом 1.

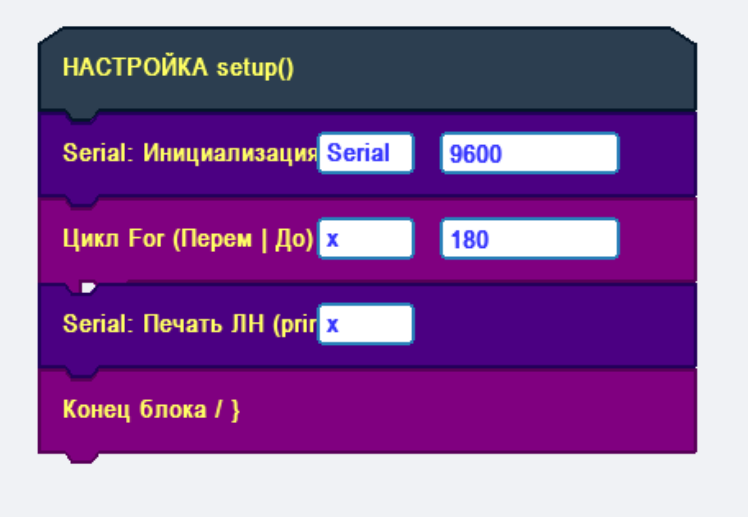


Рисунок 3.9 конечный цикл с шагом 1

Сконвертированный код в C++:

```
void setup() {
  Serial.begin(9600);
  for (int x = 0; x <= 180; x++) {
  Serial.println(x);
  }
}
```

Второй тип цикла

Конечный цикл с возможностью выбора начального и конечного значения, а также шага.

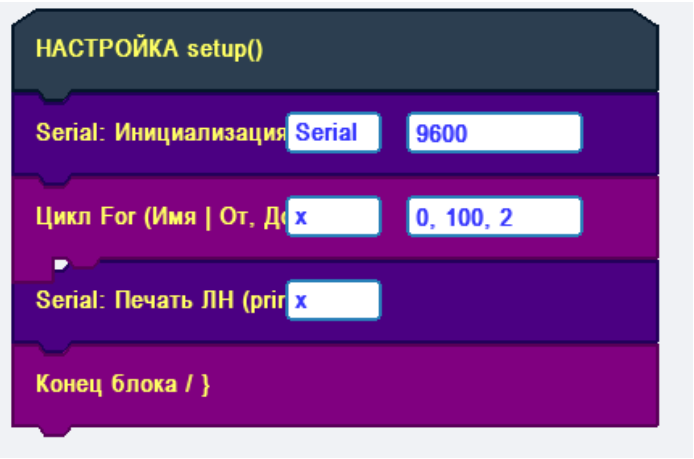
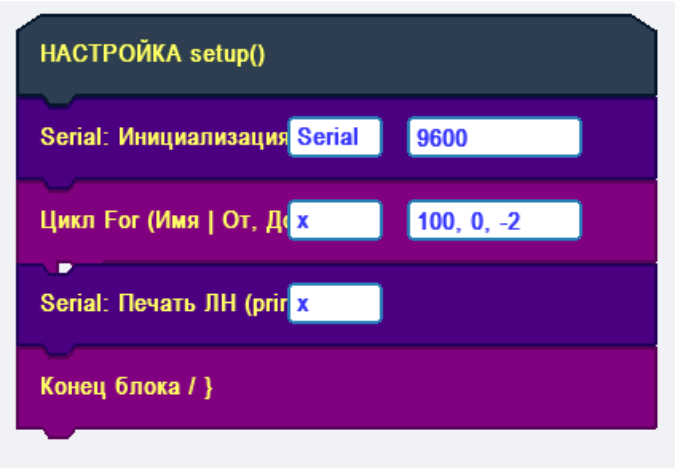


Рисунок 3.10 Цикл от меньшего к большему с шагом 2

Интерпретация его в C++

```
void setup() {
  Serial.begin(9600);
  for (int x = 0; x <= 100; x += 2) {
  Serial.println(x);
  }
}
```

Тот же блок конечного цикла с изменёнными начальными значениями и отрицательным шагом:



Преобразование кода в C++:

```
void setup() {
  Serial.begin(9600);
  for (int x = 100; x >= 0; x += -2) {
  Serial.println(x);
  }
}
```

Бесконечный цикл

Хотя функция loop уже подразумевает бесконечный цикл, но он не сможет прерваться или заново возобновится пока работает контроллер. Поэтому важно иметь конструкцию с оператором **while** под названием цикл с предусловием. С помощью неё можно проектировать небольшие независимые программы, которые будут отключать всю остальную логику, пока не выполнится процесс внутри этого цикла: передача всего пакета данных, нахождение подключённого устройства, запуск сканирования пространства и т.д.

Пример использования конструкции с while:

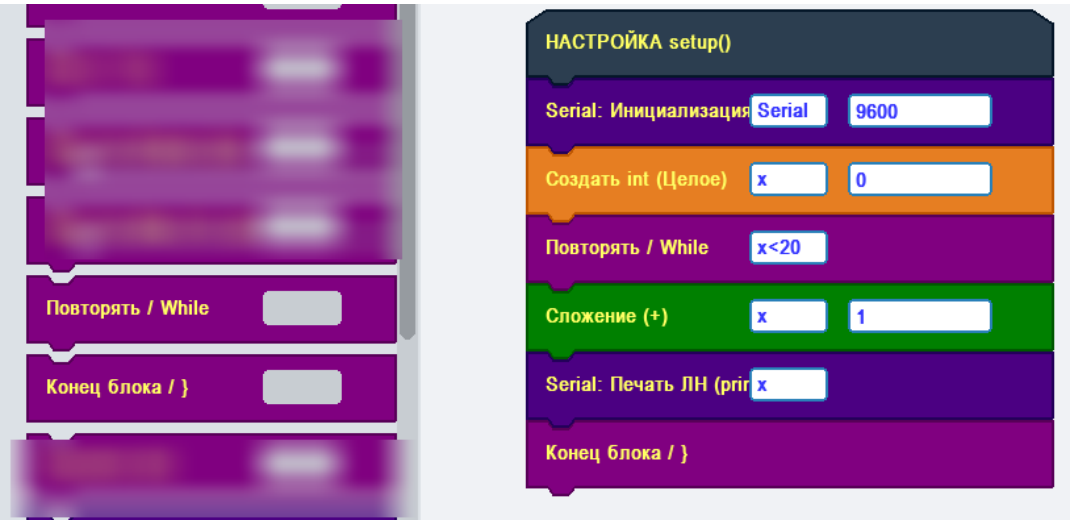


Рисунок 3.11 Цикл while

Его интерпретация на языке C++:

```
int x = 0;

void setup() {
  Serial.begin(9600);
  while(x<20) {
  x = x + 1;
  Serial.println(x);
  }
}
```

Временные интервалы

При проектировании логики робота необходимо учитывать физические особенности в скорости считывания и обработки данных, а также время выполнения того или иного физического действия. Поэтому, отдавая подобную команду необходимо прописывать некоторую задержку, которая позволит меньше нагружать контроллер и дать ему время завершить процесс.

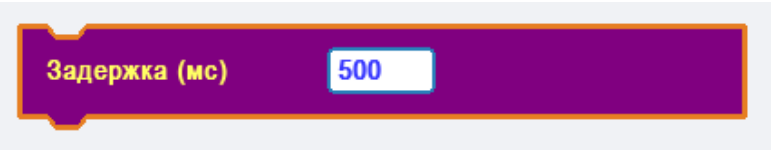


Рисунок 3.12 Задержка выполнения в миллисекундах

Интерпретацией будет:

```
delay(500);
```

Комментарии

Если ваш код будет большим и запутанным, то указание комментариев будет не лишним. Для создание таких пометок предусмотрен блок:

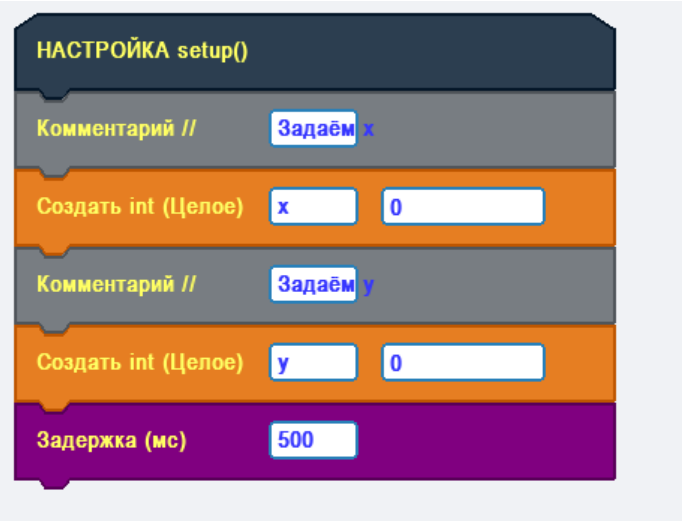


Рисунок 3.13 Применение серого блока комментария в коде

Данный блок только информирует пользователя и разработчика и не влияет на выполнение кода.

Кодировка

Для некоторых случаев необходимо преобразование чисел в определённые системы счисления. В редакторе есть команда вывода полученных данных в терминал с преобразованием в двоичную, восьмеричную, десятичную и шестнадцатеричную систему счисления.



Рисунок 3.14 Перевод в другую систему счисления

• **DEC** — Десятичный (Decimal). Используется по умолчанию, если второй параметр не указан.

• **HEX** — Шестнадцатеричный (Hexadecimal). Выводит буквы в верхнем регистре (A–F).

• **OCT** — Восьмеричный (Octal).

• **BIN** — Двоичный (Binary). Выводит число в виде нулей и единиц (без ведущих нулей).

Преобразованный код для Arduino ide:

```
Serial.begin(9600);
Serial.println(255, HEX);
```

Для преобразования подходят только числа.

Другая задача – это преобразования ASCII-кода в символ. Такой код является числом и для каждого символа будь то число, буква или какой-либо графического элемента существует свой код (нумерация).

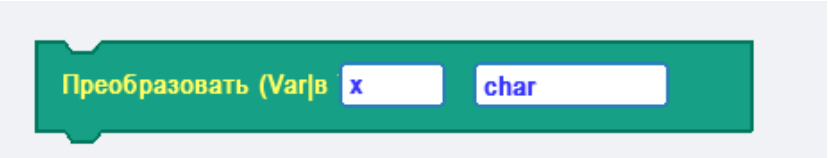


Рисунок 3.15 Преобразование в символ

Ниже представлен фрагмент кода, как это можно использовать:

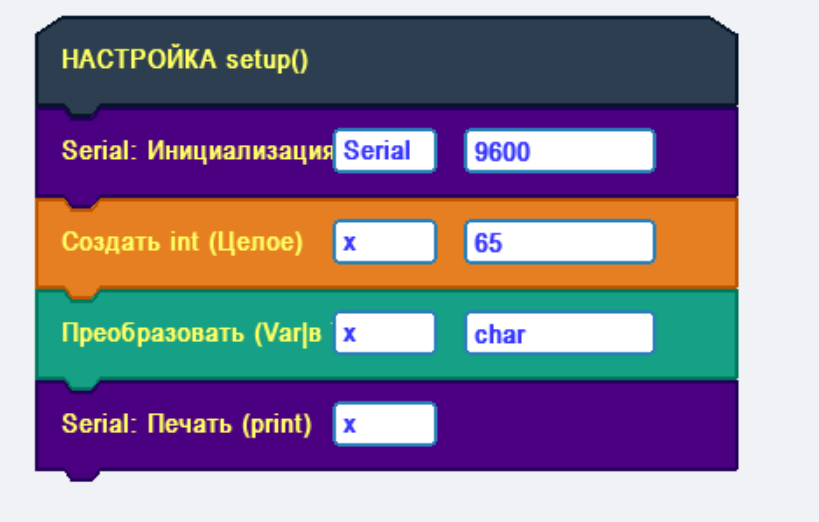


Рисунок 3.16 Получения символа из ASCII-кода

Конвертация этого кода для Arduino ide будет выглядеть так:

```
int x = 65;

void setup() {
  Serial.begin(9600);
  x = (char)x;
  Serial.print(x);
}
```

Как можете видеть, преобразовать значение можно не только в символ, но и в число, строку, или логическое значение. Для этого достаточно заменить «char» на такие команды как: int, string, bool, float.

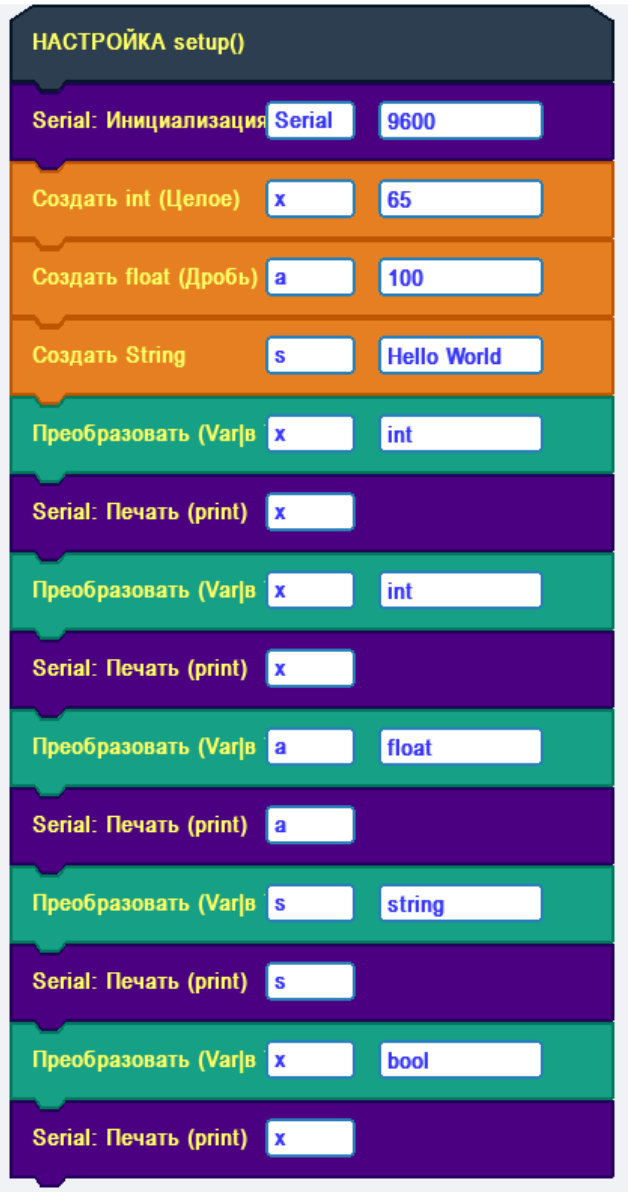


Рисунок 3.17 Преобразования

Поиск устройства по i2C

Если к контроллеру вы подключаете модуль или датчик по протоколу I2C, то очень важно узнать его адрес, чтобы корректно с ним работать.

За это отвечает один блок:

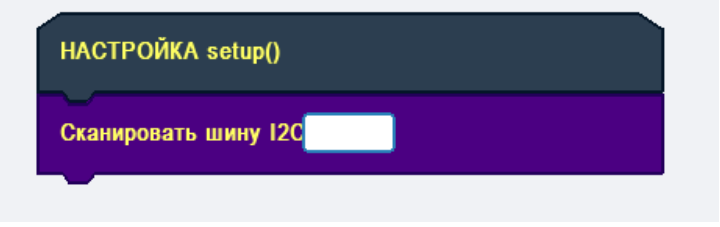


Рисунок 3.18 Блок сканирования I2C устройств в функции setup

Если сконвертировать на язык C++, то получим это:

```
#include <Wire.h>

void setup() {
  Wire.begin();
  if (!Serial) Serial.begin(9600);
  byte error, address;
  int nDevices = 0;
  Serial.println("Scanning I2C...");
  for(address = 8; address < 127; address++) {
    Wire.beginTransmission(address);
    error = Wire.endTransmission();
    if (error == 0) {
      Serial.print("I2C device found at 0x");
      if (address < 16) Serial.print("0");
      Serial.print(address, HEX);
      Serial.println(" !");
      nDevices++;
    } else if (error == 4) {
      Serial.print("Unknown error at 0x");
      if (address < 16) Serial.print("0");
      Serial.println(address, HEX);
    }
  }
  if (nDevices == 0) {
    Serial.println("No I2C devices found\n");
  } else {
    Serial.println("Scan done\n");
  }
  delay(5000); // Задержка между сканированиями
}
```

При запуске кода в терминале будет вывод адрес устройства, если оно подключено и работает корректно.

Ультразвуковой датчик расстояния

Для работы с ультразвуковым датчиком также предусмотрены блоки.

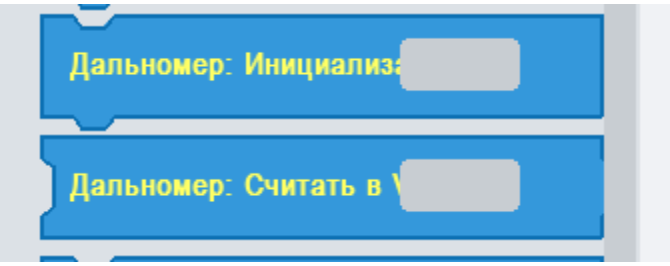


Рисунок 3.19 блоки управления датчиком расстояния

Блок «Инициализации» содержит два поля для ввода номера пинов. Первое поле это пин для Trig, а второе поле для указания пина Echo.

Ниже представлен пример программы для считывания расстояния через ультразвуковой датчик, подключённый к первому порту.

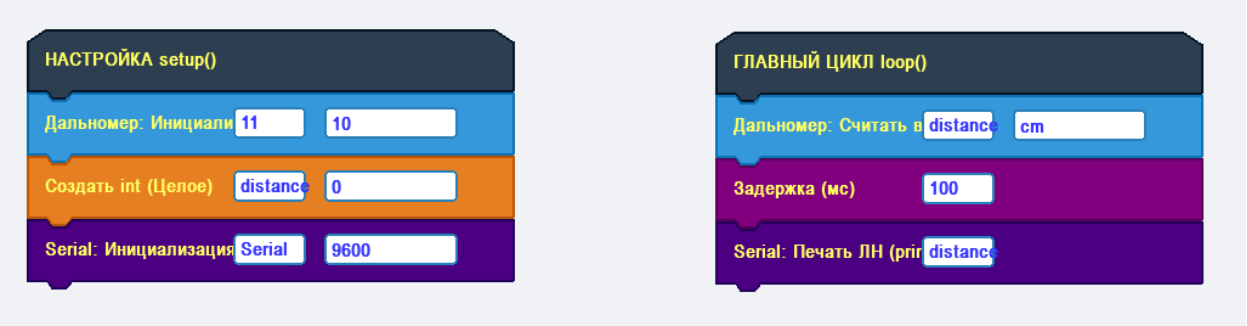


Рисунок 3.20 Программа для считывания расстояния через ультразвуковой датчик

При интеграции кода в Arduino ide мы получим вот такую запись:

```
int distance = 0;

void setup() {
  pinMode(11, OUTPUT); // Trig
  pinMode(10, INPUT);  // Echo
  Serial.begin(9600);
}

void loop() {
  digitalWrite(11, LOW);
  delayMicroseconds(2);
  digitalWrite(11, HIGH);
  delayMicroseconds(10);
  digitalWrite(11, LOW);
  distance = pulseIn(10, HIGH) / 58; // Расстояние в см
  delay(100);
  Serial.println(distance);
}
```

Указанные пины для первого порта взяты из таблицы в приложении.

Сервомотор

Ещё один элемент электроники, который требует использования библиотеки – это сервопривод. Здесь, обычно используется библиотека Servo.h.

Ниже представлены блоки для управления сервомотором:

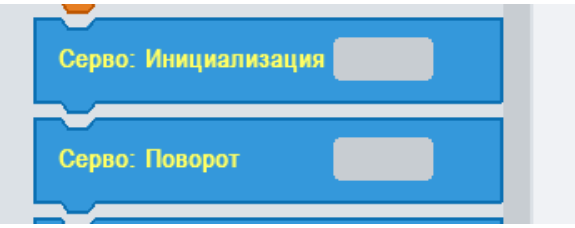


Рисунок 3.21 блоки управления сервоприводом.

Как и с датчиком расстояния необходимо инициализировать класс по работе с актуатором и указать пин подключения. Вторым блоком мы назначаем угол поворота от 0 до 180 градусов.

Ниже представлен пример реализации кода по непрерывному повороту сервопривода от 10 до 150 градусов и обратно. Сервопривод подключён ко второму порту, согласно указанному пину.

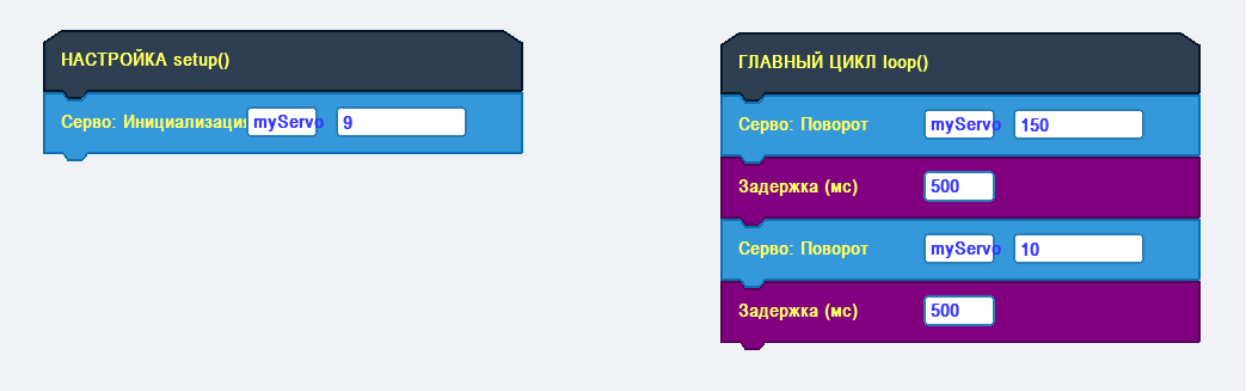


Рисунок 3.22 Программа управления сервоприводом.

Сконвертированный код выглядел бы так:

```
#include <Servo.h>
Servo myServo;

void setup() {
  myServo.attach(9);
}

void loop() {
  myServo.write(150);
  delay(500);
  myServo.write(10);
  delay(500);
}
```

Моторы с энкодерами

Для контроллеров разработаны моторы с энкодерами, которые подключаются по протоколу I2C, поэтому можно подключить одновременно 5 моторов.

Для управления ими существуют несколько командных блоков:

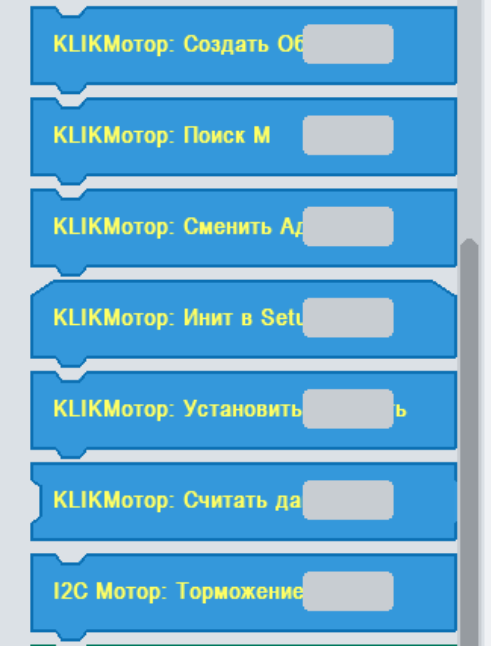


Рисунок 3.23 Блоки управления моторами с энкодерами

Если ты не знаешь точного адреса мотора, то необходимо его получить, подключив изначально только один к контроллеру. Можно воспользоваться либо командой сканера по I2C, либо блоком «Поиск мотора» совместно с блоком инициализации Serial:

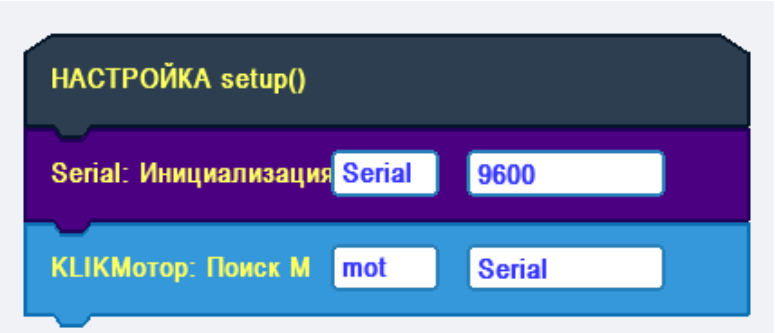


Рисунок 3.24 поиск мотора

Такой код позволит вывести в консоль адрес подключённого мотора.

Для работы с более чем одним мотором, необходимо назначить разные адреса каждому. Иначе эти актуаторы не будут синхронно работать и начнут конфликтовать.

Ниже представлена программа по смене адреса подключённому мотору. Для начала его находят, а затем переписывают адрес. Поэтому нужно использовать два командных блока. Во втором блоке укажем новый адрес, который можно менять в пределах данных интервалов:

0x07 < адрес < 0x7F

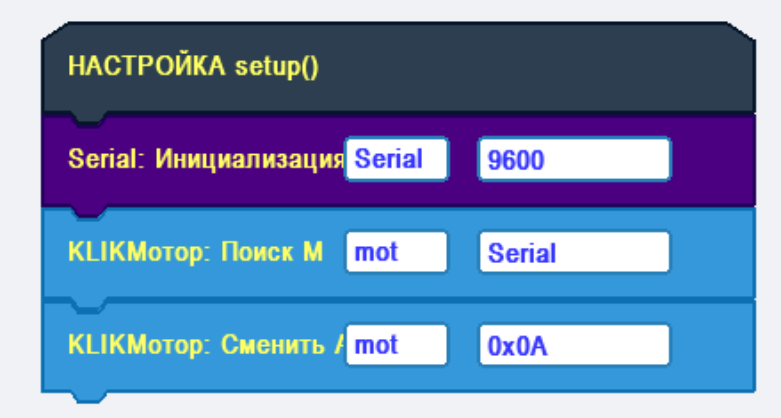


Рисунок 3.25 Поиск мотора и перезапись его адреса

Обязательно нужно учитывать то что одна переменная хранит в себе поиск мотора и дальнейший адрес.

Эти два блока интерпретируют такой код на C++:

```
#include <Wire.h>
#include <KLIK_I2C_Motor.h>
KLIK_I2C_Motor mot;
uint8_t mot_newAddress = 0x0A;

void setup() {
  Serial.begin(9600);
  delay(500);
  if (mot.begin()) {
    Serial.print("Найден мотор 0x");
    Serial.println(mot.getAddress(), HEX);
  } else {
    Serial.println("Мотор не найден!");
  }
  if (mot.begin(&Wire)) {
    if (mot.changeAddress(mot_newAddress)) {
        Serial.println(mot.getAddress(), HEX);
    } else {
        Serial.println("Адрес не изменён!");
    }
  } else {
    Serial.println("Мотор не найден!");
  }
}
```

После процедур переадресаций моторов можно приступать к написанию программ по их управлению. Зная адрес мотора создаём объект с определённым именем и адресом:

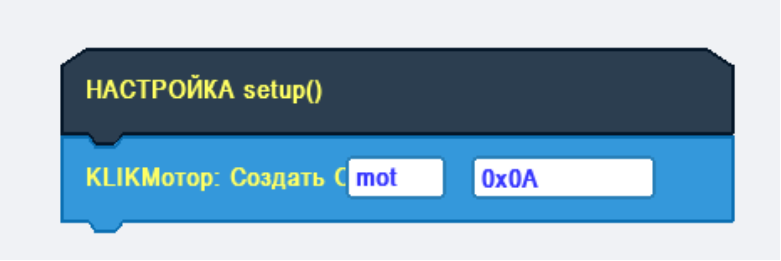


Рисунок 3.26 Переменная мотора с адресом 0x0A

Эта команда подгрузит нужные библиотеки. Вот так это будет выглядеть при интерпретации кода:

```
#include <Wire.h>

#include <KLIK_I2C_Motor.h>
KLIK_I2C_Motor mot(0x0A);

void setup() {

}
```

Далее, необходимо инициализировать данный мотор в функции setup():

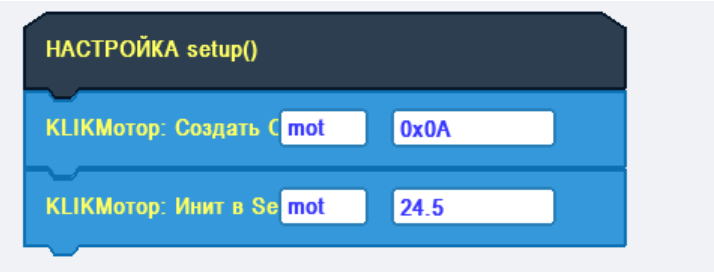


Рисунок 3.27 Инициализация мотора с адресом 0x0A

Обратите внимание, что при создании объекта мотор, что при его инициализации используется одна и та же переменная с именем `mot`. Кроме того, при инициализации нужно указать во втором поле радиус колеса в мм., которое прикреплено к мотору. Чем точнее значение, тем точнее будет движение и поворот мобильного робота.

С помощью этих блоков получается такой код на C++:

```
#include <Wire.h>
#include <KLIK_I2C_Motor.h>
KLIK_I2C_Motor mot(0x0A);

void setup() {
  mot.begin(&Wire);
  mot.radius = 24.5;
}
```

Теперь можем указать мощность мотора и направление вращения. Если значение положительное, то вращение по часовой стрелки, а если отрицательное, то против. Значения меняются в диапазоне [-100, 100].



Параметр MOT_PWM указывает, что назначается мощность мотора.

Если нужно остановить мотор, то укажи мощность равное нулю или воспользуйся одной из двух представленных блоков:

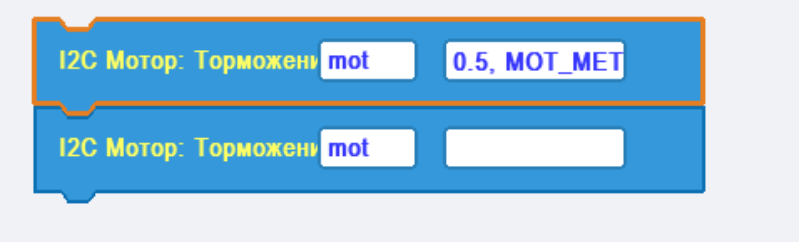


Рисунок 3.28 Два варианта остановки мотора

Согласно рис. 3.28 мы можем указать через какой промежуток пройденного расстояния или времени можно остановится или указать резкую остановку, не вводя дополнительные параметры.

Второй параметр к значению указывает, что будет считать:

MOT_REV - условие остановки задано количеством полных оборотов вала.

MOT_SEC - условие остановки задано временем.

MOT_MET - условие остановки задано расстоянием.

Если во время работы алгоритма нужно считать скорость в оборотах, м/с или в виде коэффициента мощности, то использую вот такой блок:

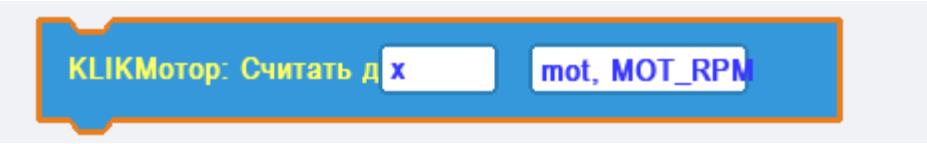


Рисунок 3.29 получаем значение скорости

Для корректной работы этого блока нужно предварительно ввести целочисленную переменную **x**.

Параметр: ТИП - тип получаемого значения:

MOT_RPM - получить реальную скорость количеством оборотов в минуту.

MOT_PWM - получить текущий коэффициент заполнения ШИМ.

MOT_M_S - получить реальную скорость в м/сек.

Ниже представлен пример алгоритма управления одним мотором с энкодером: движение по часовой стрелки, считывание скорости, остановка, движение против часовой стрелки, считывание скорости, остановка:

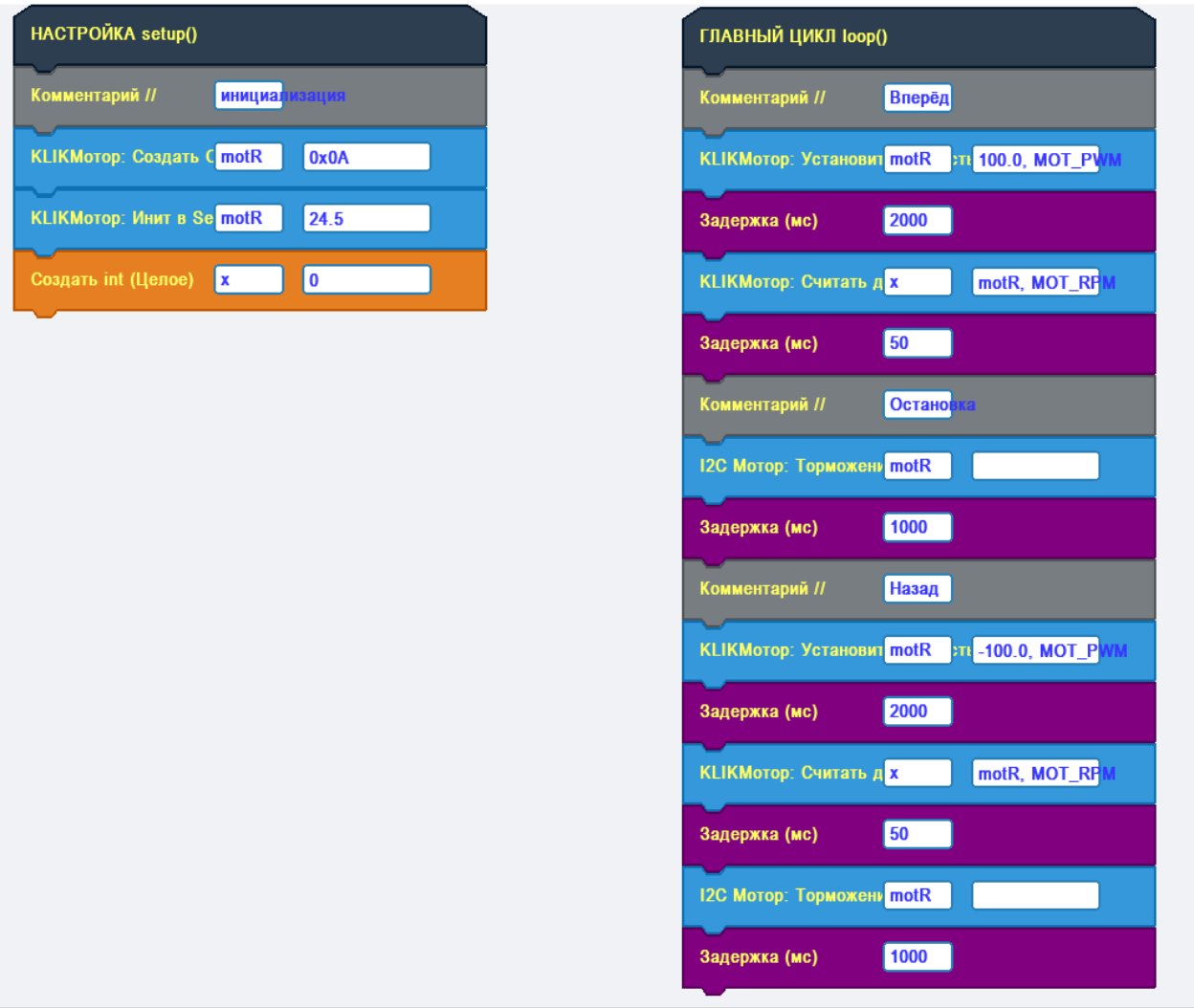


Рисунок 3.30 Пример алгоритма управления мотором с энкодером.

Этот же код, сконвертированный для Arduino ide:

```
#include <Wire.h>
#include <KLIK_I2C_Motor.h>
KLIK_I2C_Motor motR(0x0A);
int x = 0;

void setup() {
  // инициализация
  motR.begin(&Wire);
  motR.radius = 24.5;
}

void loop() {
  // Вперёд
  motR.setSpeed(100.0, MOT_PWM);
  delay(2000);
  x = motR.getSpeed(MOT_RPM);
  delay(50);
  // Остановка
  motR.setStop();
  delay(1000);
  // Назад
  motR.setSpeed(-100.0, MOT_PWM);
```

```
  delay(2000);
  x = motR.getSpeed(MOT_RPM);
  delay(50);
  motR.setStop();
  delay(1000);
}
```

IR-модуль

В наборе присутствует инфракрасный модуль с пультом дистанционного управления. Для взаимодействия с ним используют такие блок:

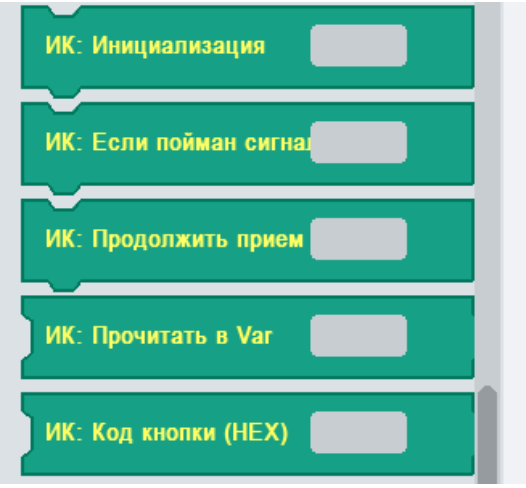


Рисунок 3.31 Блоки кода для IR - модуля

Первым делом, если используете IR-модуль, нужно инициализировать его программно.

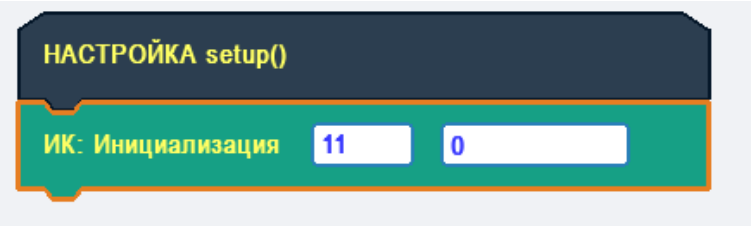


Рисунок 3.32 Инициализация IR-модуля

Для этого блока важна первое поле ввода. Здесь нужно прописать пин, к которому он подключён в зависимости от выбранного порта. В данном примере модуль подключён к первому порту.

В сконвертированном в виде этот код будет выглядеть так:

```
#include <IRremote.h>

int RECV_PIN = 11;
IRrecv irrecv(RECV_PIN);
decode_results results;

void setup() {
  Serial.begin(9600);
  irrecv.enableIRIn();
  Serial.println("IR Initialized");
}
```

Теперь нам нужно создать переменную, которая будет хранить значение сигнала с пульта. Его и принимает IR-модуль.

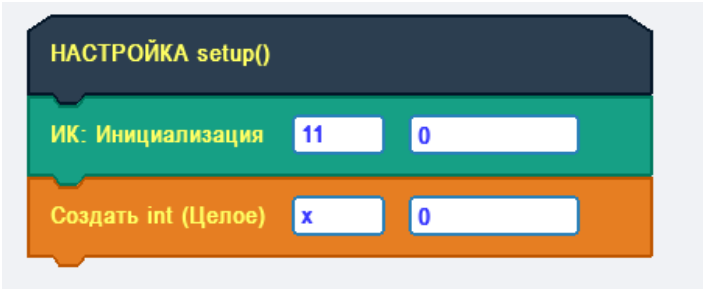


Рисунок 3.33 добавление переменной x

Осталось настроить непрерывный приём сигнала, поэтому остальная логика программы описывается в функции loop(). Добавляем блок с условием для приёма сигнала:

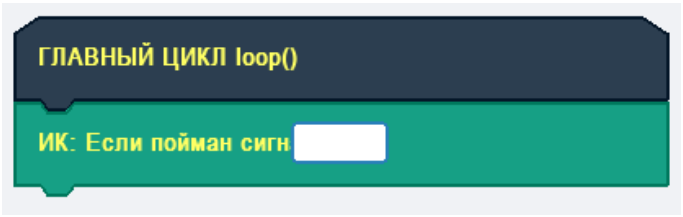


Рисунок 3.34 Блок условия для приёма сигнала

В данном блоке поле оставляют пустым так как, оно не играет никакой роли. Далее, добавляется командный блок считывания значений. В первом поле блока прописывается то имя переменной, которое было создано в setup.



Рисунок 3.35 Присваиваем полученный сигнал переменной

После присваивание значения, нужно возобновить процесс на приём нового сигнала:

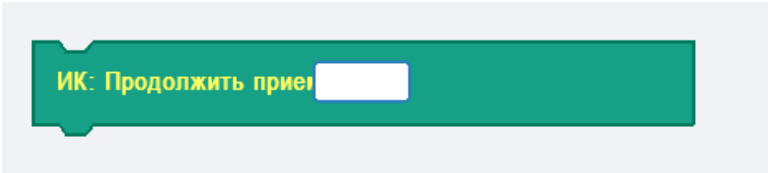


Рисунок 3.36 Возобновляем приём сигнала

В завершении, необходимо закрыть условие приёма сигнала блоком:



Рисунок 3.37 Завершения конструкции условия

В итоге мы получим вот такую конструкцию

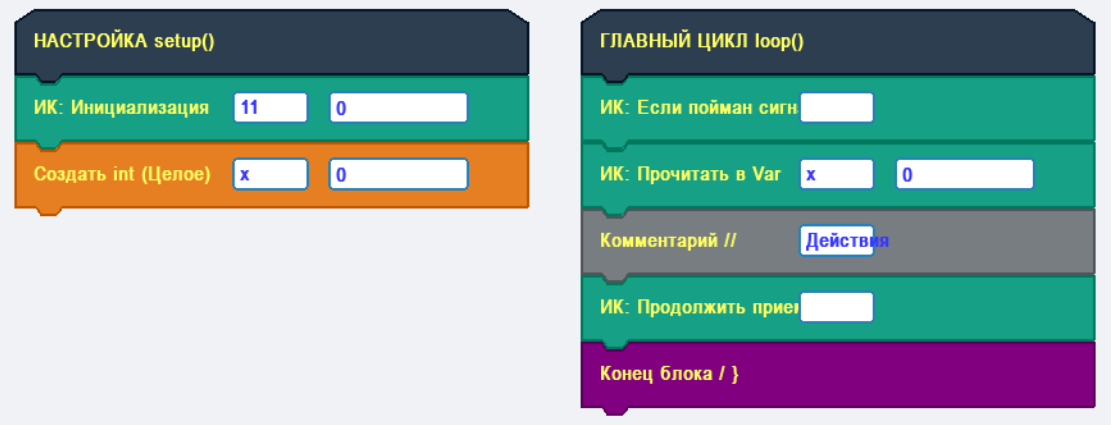


Рисунок 3.38 Общая конструкция работы с IR-модулем

Обратите внимание, что между блоками «**Чтение сигнала**» и «**Продолжить приём**» прописывают дополнительные действия с условиями. В зависимости от того какие значения контроллер получает даются определённые команды. В примере это место отмечено блоком «Комментарий»

При конвертации в язык C++ - это выглядело бы так:

```
#include <IRremote.h>
int RECV_PIN = 11;
IRrecv irrecv(RECV_PIN);
decode_results results;
int x = 0;

void setup() {
  Serial.begin(9600);
  irrecv.enableIRIn();
  Serial.println("IR Initialized");
}

void loop() {
  if (irrecv.decode(&results)) {
  x = results.value;
  // Действия
  irrecv.resume();
  }
}
```

Пример сочетания IR-модуля и двух DC-моторов. Значения сигналов, которые сравниваются в условиях могут отличаться от ваших. Всегда предварительно выводите сигналы через Serial.

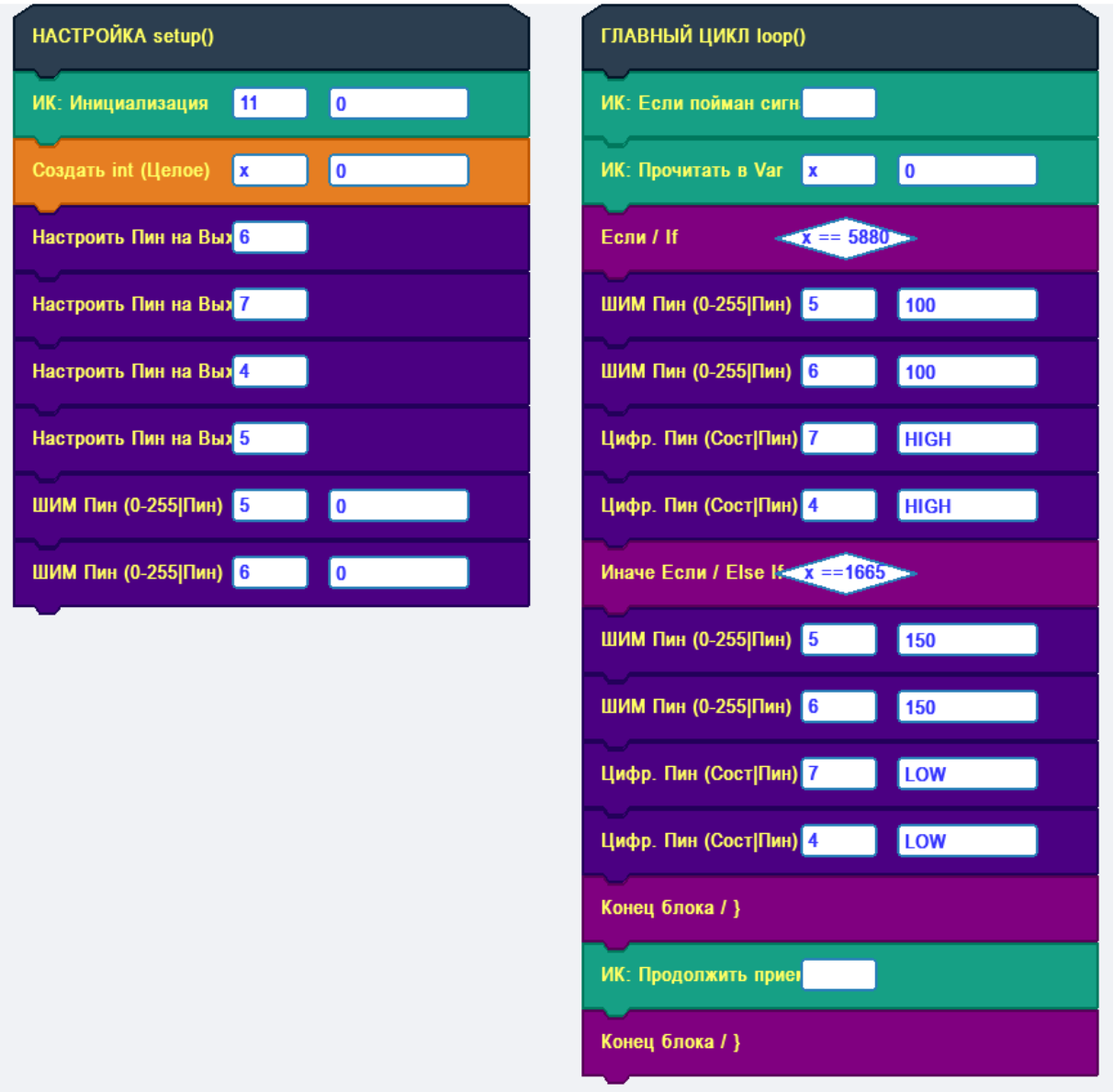


Рисунок 3.39 Пример управления моторами через IR-модуль

Bluetooth

Ещё один способ дистанционного управления, которое поддерживает набор Клик, является связь по Bluetooth. С помощью этого типа радиосвязи в качестве контроллера можно использовать мобильные устройства (смартфоны и планшеты).

Подключите к 4 порту контроллера Bluetooth модуль и установите на телефон приложение Bluetooth Terminal или схожее по типу связи. Включите на мобильном устройстве Bluetooth и найдите модуль. После проведите сопряжение (пароль связи 1234).

Для взаимодействия телефона и контроллера нужно применить вот такой блок:



Рисунок 3.40 Блок настройки логики взаимодействия по Bluetooth

Этот командный блок размещается в двух местах в setup и в loop. В первом случае он нужен для указания однократного действия сброса питания, а второй для прописывания логики приёма и записи радиосигнала. Радиосигнал передаётся символами в кодировке **ASCII** в десятичном формате (смотри приложение 2), которые нужно записать в переменную. Поэтому блок создания переменной будет **целочисленный**. В первом поле прописывается пин, отвечающий за питание портов контроллера. В нашем случае это A4. Во втором поле прописывается переменная, которая будет хранить полученные команды. Также обязательно нужно инициализировать Serial, так как по технологии будут передаваться данные.

Ниже представлена базовая конструкция настройки по Bluetooth:

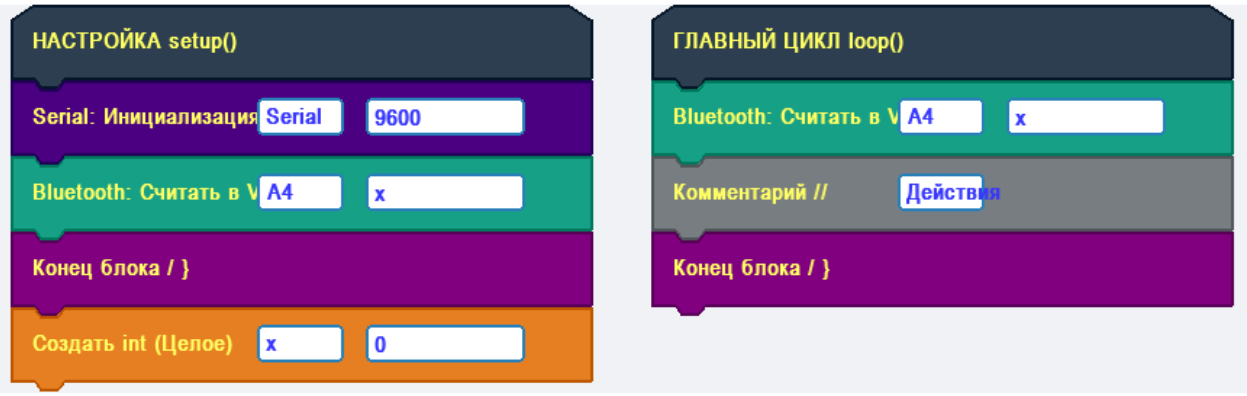


Рисунок 3.41 Основа логики Bluetooth взаимодействия.

Там, где указан блок «Комментарий» вставляются условия на проверку полученного символа и указания действия, если есть совпадения.

Для Arduino ide этот код выглядел бы так:

```
int ser_enable = A4;
int x = 0;

void setup() {
  Serial.begin(9600);
  pinMode(ser_enable, OUTPUT);
  digitalWrite(ser_enable, LOW);
  delay(1);
  digitalWrite(ser_enable, HIGH);
  if (Serial.available())
  {
     x = Serial.read();
  }
}

void loop() {
  if (Serial.available())
  {
     x = Serial.read();
  // Действия
  }
}
```

Ниже представлен пример управление одним DC-мотором с помощью Bluetooth. Значения ASCII 119 и 97 отвечают латинским символам: **w** и **a.**

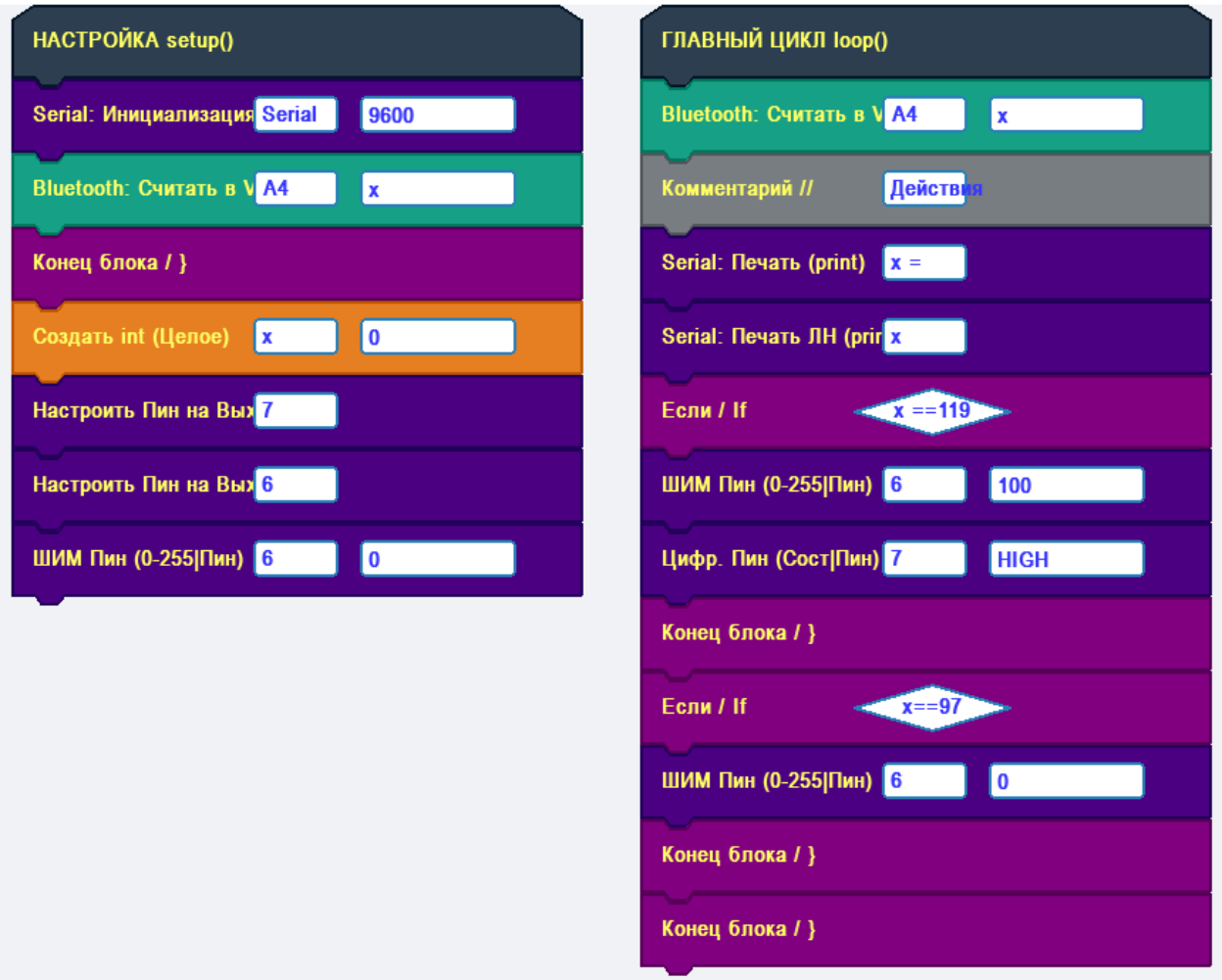


Рисунок 3.42 Пример управление мотором по Bluetooth

Этот же код на C++:

```
int ser_enable = A4;
int x = 0;

void setup() {
  Serial.begin(9600);
  pinMode(ser_enable, OUTPUT);
  digitalWrite(ser_enable, LOW);
  delay(1);
  digitalWrite(ser_enable, HIGH);
  if (Serial.available())
  {
     x = Serial.read();
  }
  pinMode(7, OUTPUT);
  pinMode(6, OUTPUT);
  analogWrite(6, 0);
}

void loop() {
  if (Serial.available())
  {
     x = Serial.read();
  // Действия
  Serial.print("x = ");
```

```
  Serial.println(x);
  if (x ==119) {
  analogWrite(6, 100);
  digitalWrite(7, HIGH);
  }
  if(x==97) {
  analogWrite(6, 0);
  }
  }
}
```

Датчик цвета

В наборе идёт датчик цвета на микрочипе TCS34725, поэтому для корректного считывания данных используется особая библиотека, инициализировать которую можно с помощью этого блока:

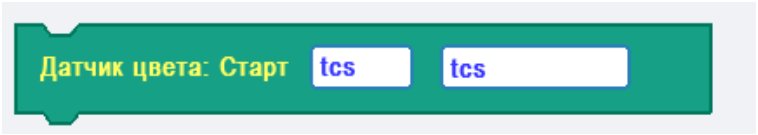


Рисунок 3.43 Инициализация библиотеки датчика цвета

Этот блок нужно использовать совместно с блоком инициализации Serial.



Рисунок 3.44 Загрузка библиотеки датчика и подключение Serial

Это сочетание даёт вот такой код на C++

```
#include <Wire.h>

#include "Adafruit_TCS34725.h"
Adafruit_TCS34725 tcs = Adafruit_TCS34725(TCS34725_INTEGRATIONTIME_50MS,
TCS34725_GAIN_4X);

void setup() {
  Serial.begin(9600);
  if (!tcs.begin()) { Serial.println("TCS34725 Error!"); }
}
```

Далее, нам нужно считывать значения цвета в формате RGB. Поэтому нужно создать три числовые переменные для записи этих значений цвета:

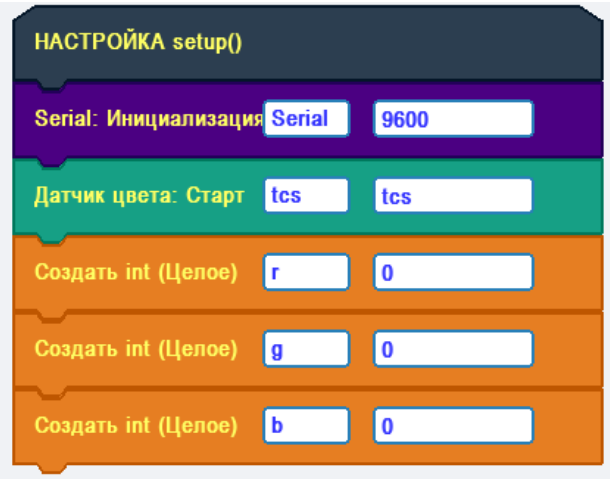


Рисунок 3.45 Вводим три переменные для хранения цвета в формате RGB

Преобразовывать полученные данные в значение цвета в диапазоне от 0 до 255 будет вот такой блок:



Рисунок 3.46 конвертирование значения в формат RGB

Таких блоков используем в количестве трёх штук. В левом поле вводим один из цветов формата: **R**, **G** или **B**. В правом поле ставим имя одной из созданных переменных. Значения выведем с помощью Serial.



Рисунок 3.47 Полный шаблон кода для датчика цвета.

При конвертации кода получим вот такую запись:

```
#include <Wire.h>
#include "Adafruit_TCS34725.h"
Adafruit_TCS34725 tcs = Adafruit_TCS34725(TCS34725_INTEGRATIONTIME_50MS,
TCS34725_GAIN_4X);
int r = 0;
int g = 0;
int b = 0;

void setup() {
  Serial.begin(9600);
  if (!tcs.begin()) { Serial.println("TCS34725 Error!"); }
}

void loop() {
  {
    uint16_t tcs_r, tcs_g, tcs_b, tcs_c;
    tcs.getRawData(&tcs_r, &tcs_g, &tcs_b, &tcs_c);
    r = tcs_r;
  }
  {
    uint16_t tcs_r, tcs_g, tcs_b, tcs_c;
    tcs.getRawData(&tcs_r, &tcs_g, &tcs_b, &tcs_c);
    g = tcs_g;
  }
  {
    uint16_t tcs_r, tcs_g, tcs_b, tcs_c;
    tcs.getRawData(&tcs_r, &tcs_g, &tcs_b, &tcs_c);
    b = tcs_b;
  }
  Serial.print("R =");
  Serial.println(r);
  Serial.print("G = ");
  Serial.println(g);
  Serial.print("B = ");
  Serial.println(b);
  delay(200);
}
```

Дисплей 16x8

Для данного контроллера разработан светодиодный дисплей с разрешением 16x8. Для управления им созданы блоки:

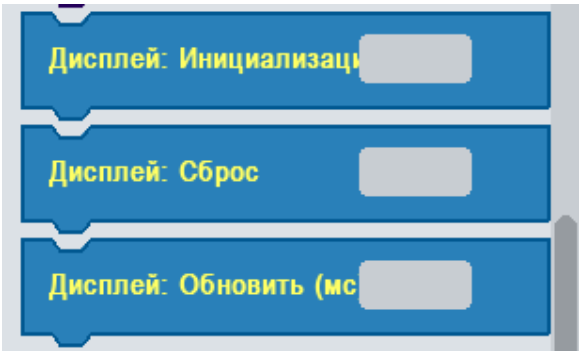


Рисунок 3.48 Командные блоки для дисплея

Первым делом, нужно инициализировать подключённое устройство, путём загрузки нужных библиотек и нахождения по зарезервированному адресу 0x70.

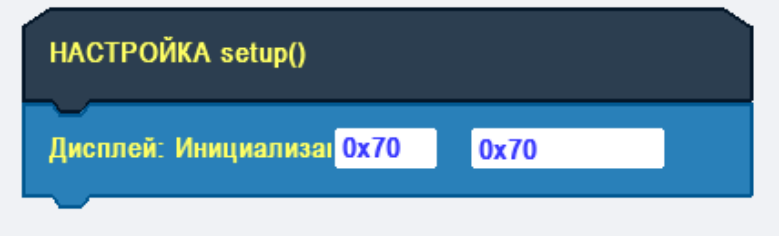


Рисунок 3.49 указанные значения в полях не меняем.

Данный блок интерпретирует такой код :

```
const int sda_pin = A4;
const int scl_pin = A5;
uint16_t displaybuffer[8];
void i2c_start() {
digitalWrite(sda_pin, HIGH); digitalWrite(scl_pin, HIGH);
delayMicroseconds(5);
digitalWrite(sda_pin, LOW); delayMicroseconds(5); digitalWrite(scl_pin, LOW);
}
void i2c_stop() {
digitalWrite(sda_pin, LOW); digitalWrite(scl_pin, HIGH);
delayMicroseconds(5);
digitalWrite(sda_pin, HIGH); delayMicroseconds(5);
}
void i2c_write(uint8_t data) {
for (uint8_t i = 0; i < 8; i++) {
digitalWrite(sda_pin, (data & 0x80) ? HIGH : LOW); data <<= 1;
delayMicroseconds(5);
digitalWrite(scl_pin, HIGH); delayMicroseconds(5); digitalWrite(scl_pin,
LOW);
}
digitalWrite(sda_pin, HIGH); delayMicroseconds(5); digitalWrite(scl_pin,
HIGH);
delayMicroseconds(5); digitalWrite(scl_pin, LOW);
}
void matrix_begin(uint8_t addr) {
pinMode(sda_pin, OUTPUT); pinMode(scl_pin, OUTPUT);
i2c_start(); i2c_write(addr << 1); i2c_write(0x21); i2c_stop(); delay(10);
i2c_start(); i2c_write(addr << 1); i2c_write(0x81); i2c_stop(); delay(10);
i2c_start(); i2c_write(addr << 1); i2c_write(0xE5); i2c_stop();
}
void matrix_clear() {
for (uint8_t i = 0; i < 8; i++) displaybuffer[i] = 0;
}
void matrix_writeDisplay(uint8_t addr) {
i2c_start(); i2c_write(addr << 1); i2c_write(0x00);

for (uint8_t i = 0; i < 8; i++) {
uint16_t data = displaybuffer[i];
i2c_write(data & 0xFF);
i2c_write((data >> 8) & 0xFF);
}
i2c_stop();
}

void setup() {
  matrix_begin(0x70);
  matrix_clear();
}
```

После, необходимо сбросить дисплей, чтобы очистить его, если уже указаны были какие-то изображения.

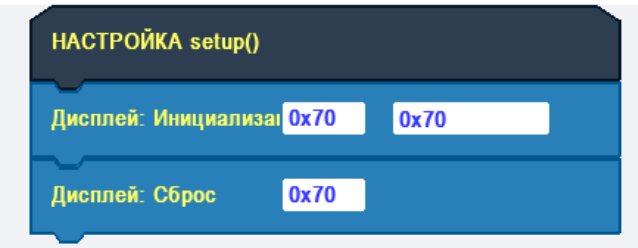


Рисунок 3.50 Инициализация и принудительная очистка

Для непосредственного создания рисунка, необходимо воспользоваться таким блоком:



Рисунок 3.51 Блок для создания изображения

Добавив его в код, кликните по нем правой кнопкой мыши, чтобы открылось окно редактора дисплея:

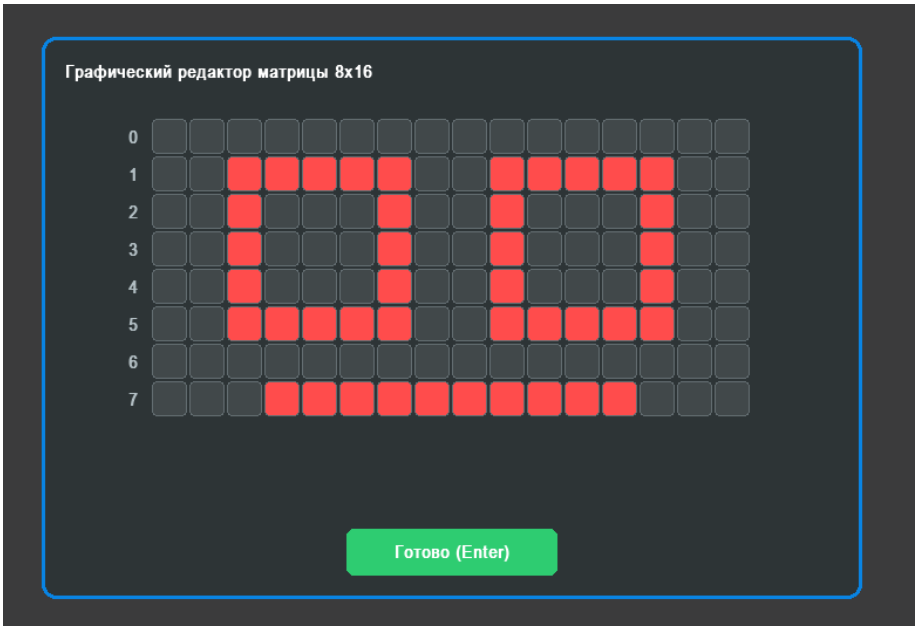


Рисунок 3.52 Окно рисования на дисплее

С помощью нажатия ЛКМ закрашиваем нужную ячейку и тем самым создаём изображения. По окончанию редактуры кликните по кнопке «Готово».

На языке C++ этот рисунок будет программироваться так:

```
// --- Генерация графического рисунка матрицы (С коррекцией осей) ---
  displaybuffer[0] = 0B1000000000000000;
  displaybuffer[1] = 0B1011111000000000;
  displaybuffer[2] = 0B1010001000111110;
  displaybuffer[3] = 0B1010001010100010;
  displaybuffer[4] = 0B1010001010100010;
  displaybuffer[5] = 0B0011111010100010;
```

```
displaybuffer[6] = 0B0000000010111110;
displaybuffer[7] = 0B0000000010000000;
matrix_writeDisplay(0x70);
delay(1000);
```

Используя этот блок и блок задержки, можно создать анимированные переходы изображений на дисплее.

Двойной датчик линии, кнопка

Эти датчики и модуль реализуется исключительно командами по управлению пинами

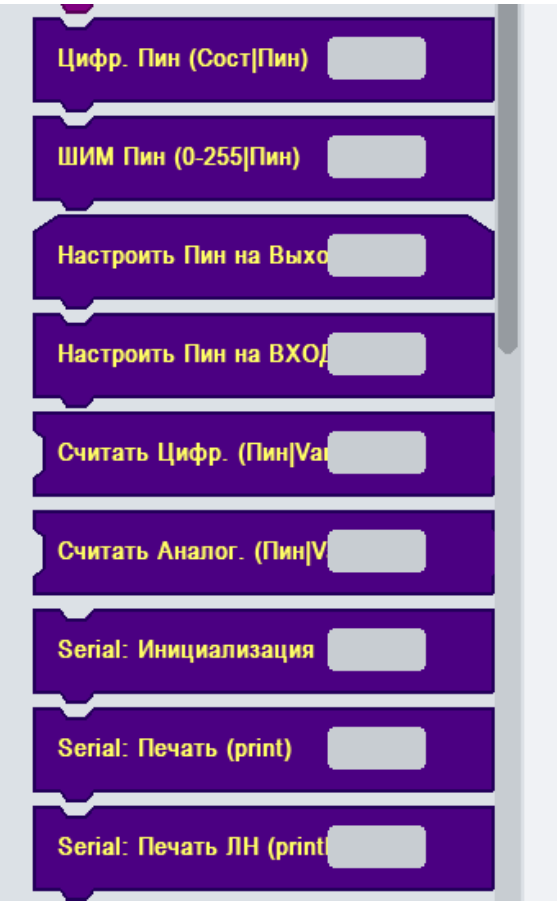


Рисунок 3.53 команды управления пинами

С помощью этого подхода можно управлять большим набором датчиков и модулей, завязанных на передачу аналоговых данных или приёму исключительно цифровых сигналов (0 или 1).

Под управления попадает такие элементы как: лазерный модуль, датчик влажности почвы, датчик освещённости, датчик уровня воды, светодиоды, кнопки, датчики линии, датчик огня, электромагнитное реле и т.д.

Функции

Редактор поддерживает создание собственных функций, для многократного использования.

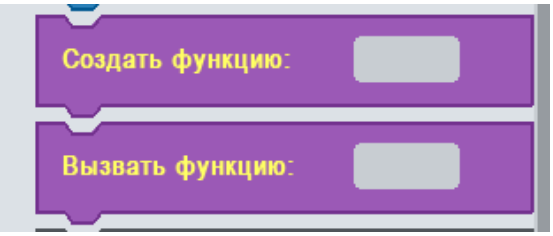


Рисунок 3.54 Блоки по работе с функциями

Из названий блоков видно, что для создания функции нужно использовать соответствующий блок, под которым будем крепить другие блоки, создавая алгоритм. После этого вызываем нашу функцию в основном коде, используя второй блок.

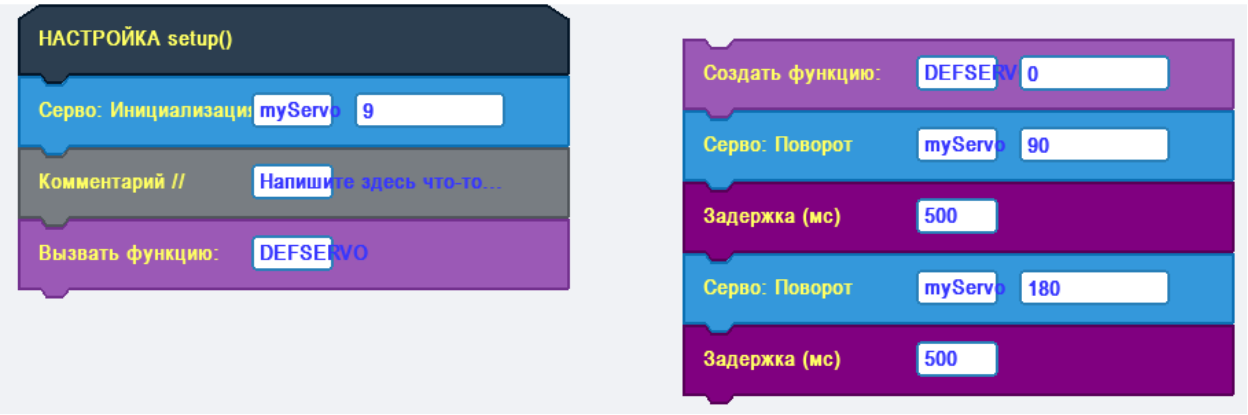


Рисунок 3.55 Пример реализации функции с именем DEFSERVO и её вызова.

Глава 4 Работа с KLIK EDITOR MICROPYTHON

Структура интерфейса

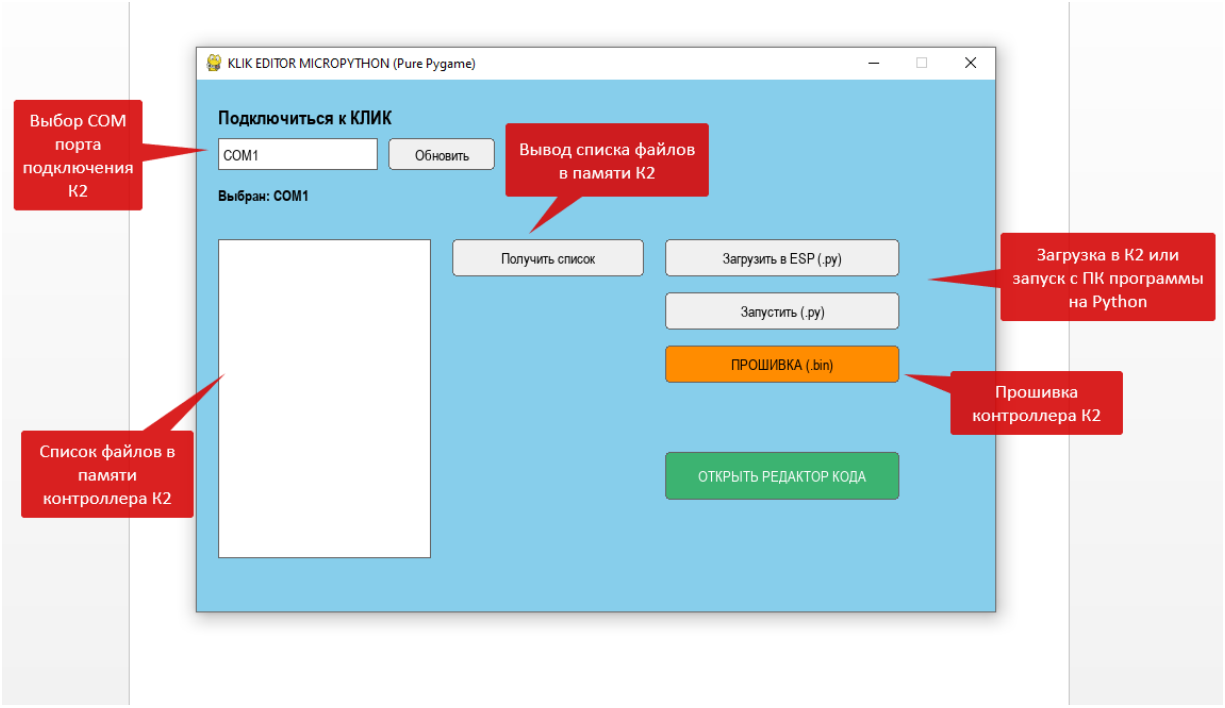


Рисунок 4.1 Структура интерфейса

С помощью этой программы мы можем загружать или выгружать файлы в контроллер К2. Программы для данного контроллера, записываются на языке Python, а точнее на microPython.

Если на ПК нет Python или редактора кода, то можете воспользоваться встроенным редактором, для просмотра, редактирования или создания файлов с расширением .py.

Программирование

Для работы со встроенным дисплеем, кнопками, устройствами, подключаемыми по i2c необходимо предварительно загрузить в память контроллера К2, если их изначально там не было.

- ir_rx
- i2c_device.py
- i2C_Scaner.py
- ssd1306.py

Основной модуль microPython, который идёт совместно с прошивкой это **machine**. С его помощью можно получить доступ к пинам контроллера к2, а также взаимодействовать с устройствами по i2c.

Пример программы представлены в репозитарии на **GitHub**:

Необходимые **модули** также представлены там

Здесь вы можете изучить документацию **mashine**.

Для того чтобы программировать на Python, необходимо изучить материал. Предлагаю вам список литературы как для детей, так и для учителей:

- Python. Великое программирование в Minecraft. Андрей Корягин, Алиса Корягина. – Изд. 6, испр. и допол. – Ростов н/Д: Феникс, 2027.
- Python. Погружение в математику с Minecraft. Андрей Корягин, Алиса Корягина. – Изд. 2-е, испр. и допол. – Ростов н/Д: Феникс, 2025.
- Азбука Python. Подробный самоучитель для школьников и студентов / А. В. Корягина. — М. : Лаборатория знаний, 2027.

Предлагаю цикл **видеоуроков** по программированию на microPython для ESP32

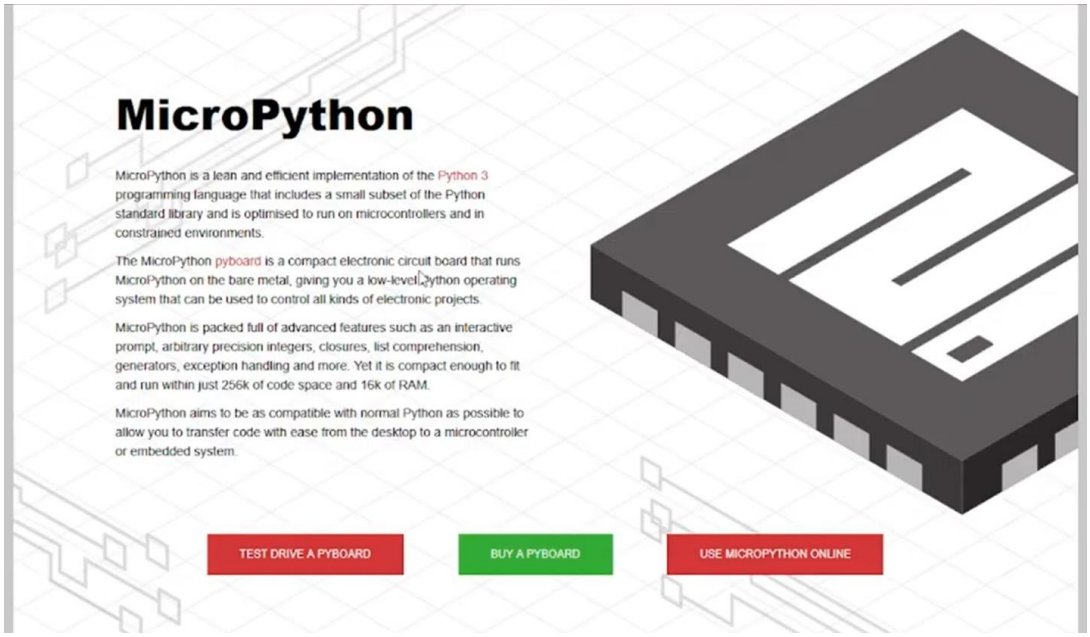


Рисунок 4.2 видеоуроки по microPython

Приложение 1

Порт Блок А	Ардуино (Arduino) (вход/выход)	Примечания. Рекомендуемые устройства и датчики для подключения.
1	D10, D11, SDA, SCL	Серводвигатель, Датчик расстояния, Датчик ИК, Датчик линии, Датчик цвета. Датчик стоп, Цифровой вход/ выход Ардуино, I2C интерфейс.
2	D9, D3, SDA, SCL	Серводвигатель, Датчик ИК, Датчик цвета. Датчик стоп, Цифровой вход/ выход Ардуино, I2C интерфейс.
3	A3, A2, SDA, SCL	Датчик цвета, Аналоговый вход Ардуино, Дополнительные датчики Температуры, Влажности и.т.п. I2C интерфейс
4	D0(Rx), D1(Tx), SDA, SCL	Последовательный порт, Датчик BlueTooth, Серводвигатель, Датчик расстояния, Датчик ИК, Датчик линии, Датчик цвета. Датчик стоп, Цифровой вход/ выход Ардуино, I2C интерфейс.
5	D13, D12, SDA, SCL	Серводвигатель, Датчик расстояния, Датчик ИК, Датчик линии, Датчик цвета, Датчик стоп, Цифровой вход/ выход Ардуино, I2C интерфейс.
6	D2, D8, SDA, SCL	Серводвигатель, Датчик расстояния, Датчик ИК, Датчик линии, Датчик цвета. Датчик стоп, Цифровой вход/ выход Ардуино, I2C интерфейс.
M 1	D6, D7	Мотор 1. Для управления мотора используется цифровой выход Ардуино. D6 - скорость, D7 - направление
M 2	D4, D5	Мотор 2. Для управления мотора используется цифровой выход Ардуино. D5 - скорость, D4 - направление

Buzzer(Динамик) - выход A0

Включение последовательного порта - выход A4.

Приложение 2

Dec (Десятичный)	Char (Символ)	Описание / Назначение
0	NUL	Нулевой символ
9	HT	Горизонтальная табуляция (Tab)
10	LF	Перевод строки (Line Feed)
13	CR	Возврат каретки (Carriage Return)
32		Пробел (Space)
33	!	Восклицательный знак
48–57	0–9	Цифры от 0 до 9
65–90	A–Z	Заглавные латинские буквы
97–122	a–z	Строчные латинские буквы
127	DEL	Удаление (Delete)